



机器人学实验报告

Lab 2:BP + PID

专 业： 人工智能

姓 名： 马承乾

学 号： 22920212204180

实验日期： 2023.12.8

指导教师： 晁飞

目录

1	实验目的	3
2	实验要求	3
3	实验流程	3
3.1	最佳效果展示	3
3.2	BP 网络的学习速度和冲量的影响	4
3.2.1	修改学习速率:	4
3.2.2	学习率的分析	5
3.2.3	修改冲量:	6
3.2.4	分析冲量	7
3.3	尝试初始权值	7
3.3.1	全部初始化为 0	7
3.3.2	全部初始化为 0.5	8
3.3.3	全部初始化为 -0.5	9
3.3.4	初始化为随机数	10
3.3.5	初始化分析	11
3.4	尝试 H 个数	11
3.4.1	神经元个数为 5:	12
3.4.2	神经元个数为 20:	12
3.4.3	神经元个数为 50:	13
3.4.4	神经元个数为 100:	14
3.4.5	神经元个数分析	14
3.5	当跟踪曲线是 $\%rin(k)=\sin(2*3.1415926*time(k))$, 是否可以收敛	15
3.5.1	只改变跟踪曲线	15
3.5.2	调参	15
3.5.3	拟合效果分析	16
3.6	两层隐含层	16
3.6.1	变量间关系及运行流程如下:	16
3.6.2	初步效果展示	18
3.6.3	调参	18
3.6.4	相关分析:	19
4	实验总结	19
5	实验心得	20
6	附录	20

1 实验目的

- 熟悉神经网络的特征、结构及学习算法。
- 通过实验掌握神经网络自整定 PID 的工作原理。
- 了解神经网络的结构对控制效果的影响。
- 掌握用 Matlab 实现神经网络控制系统仿真的方法。

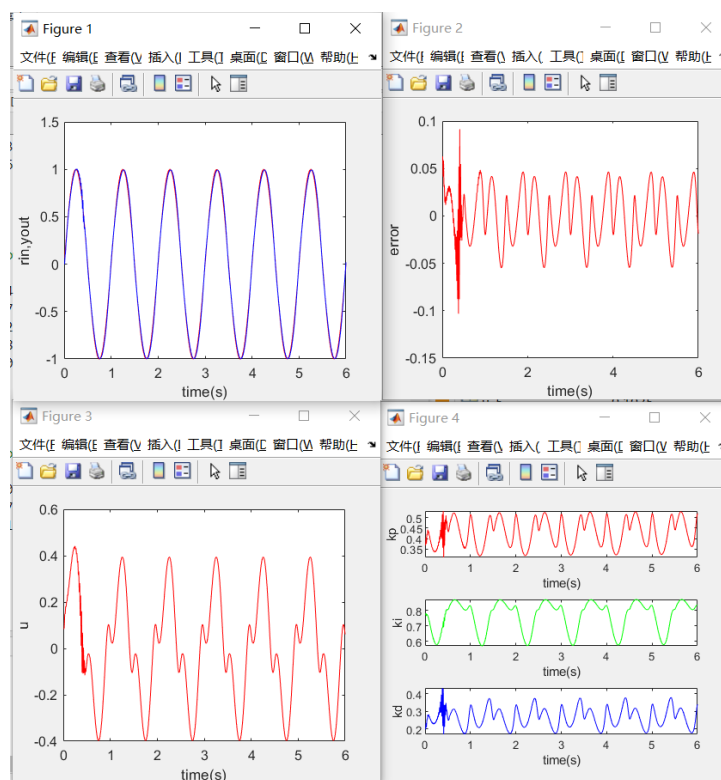
2 实验要求

- 任务一：BP 网络的学习速度和冲量的影响
- 任务二：尝试初始权值全是 0, 0.5, -0.5, 随机
- 任务三：尝试 H 个数， 5,20,50,100……
- 任务四：当跟踪曲线是 $\%rin(k)=\sin(2*3.1415926*time(k))$ ，是否可以收敛
- 学有余力的同学：两层隐含层

3 实验流程

3.1 最佳效果展示

双隐藏层+较困难的三角函数跟踪曲线+保留测试过程中的优良权重矩阵+反复测试得到的参数，得到效果如下：

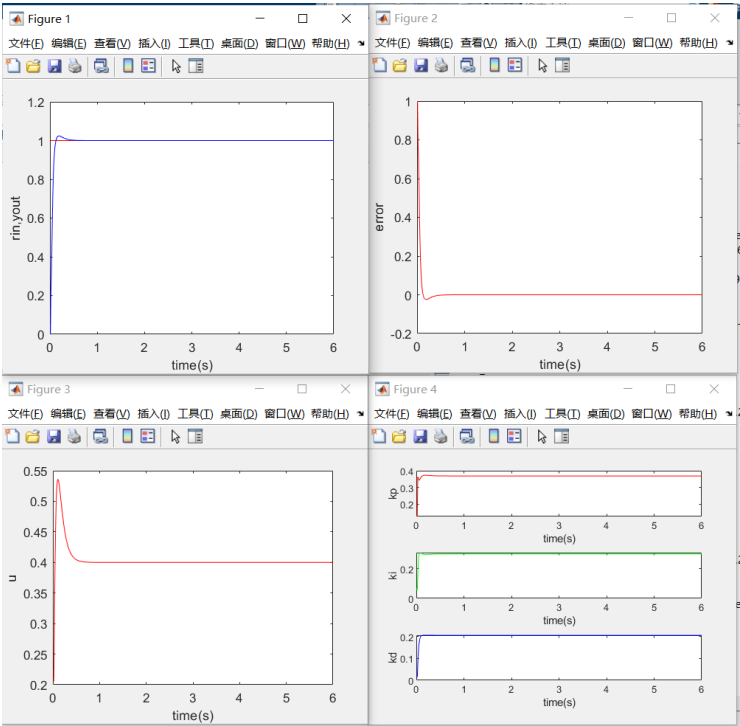


最大超调量	调节时间(s)	稳态误差
0.5	0.45	2.7%

3.2 BP 网络的学习速度和冲量的影响

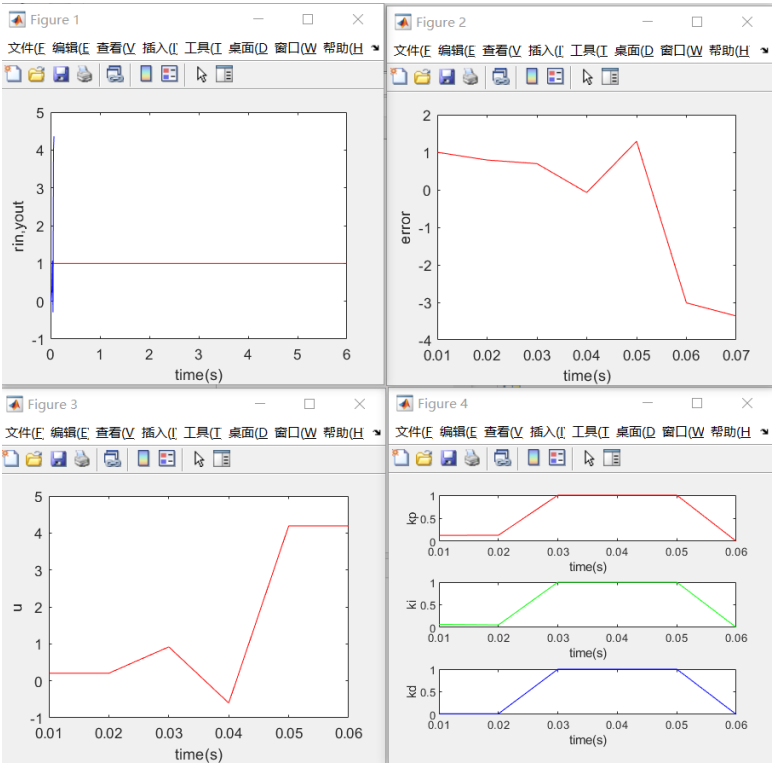
3.2.1 修改学习速率：

✚ 将学习率设置为 0.1，得到效果如下：



最大超调量	调节时间(s)	稳态误差
3.4%	0.45	3.3%

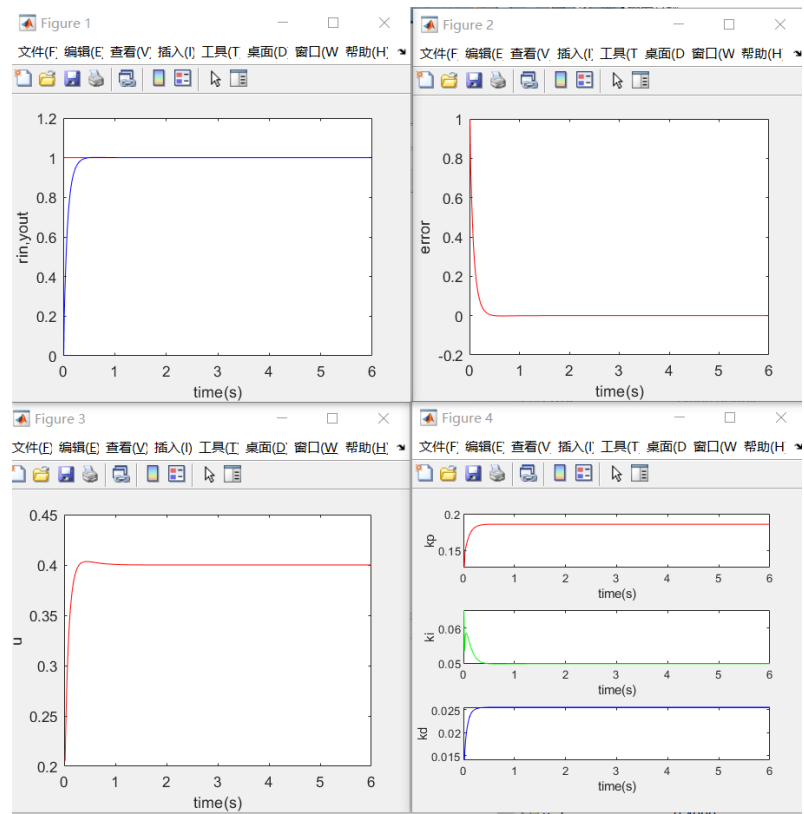
✚ 将学习率增大为 0.5：



观察到当学习率变大时，模型的拟合效果变差。

最大超调量	调节时间(s)	稳态误差
未到达稳态	未到达稳态	未到达稳态

将学习率缩小为 0.01:



此时模型学习比较谨慎，因此效果比较优秀。

最大超调量	调节时间(s)	稳态误差
2.2%	0.37	1.2%

3.2.2 学习率的分析

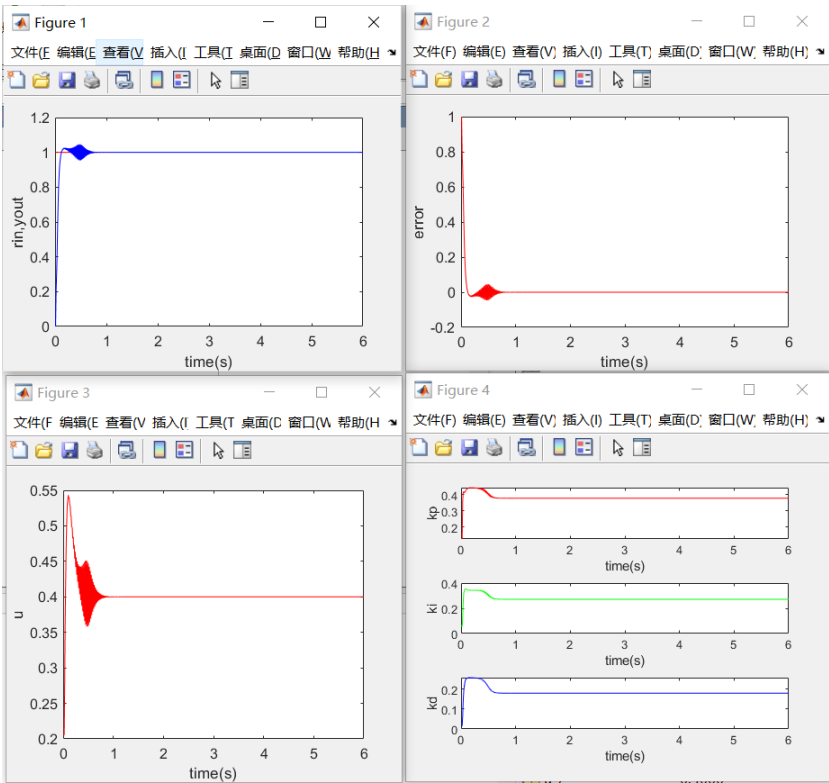
学习率是在神经网络的训练过程中用于调整权重更新步长的一个超参数。控制了神经网络在每次迭代中对模型参数进行调整的大小。学习率的选择对于训练的效果和速度都有很大的影响。

不同情况下学习率的作用可以总结如下：

- 较大的学习率：
 - 优势：训练速度较快。
 - 劣势：容易导致不稳定的训练，难以收敛。
- 适中的学习率：
 - 优势：较好的平衡训练速度和稳定性。
 - 劣势：仍可能在某些情况下陷入局部最优解。
- 较小的学习率：
 - 优势：训练相对稳定，收敛到较好的解。
 - 劣势：训练速度较慢，可能需要更多的迭代。

3.2.3 修改冲量：

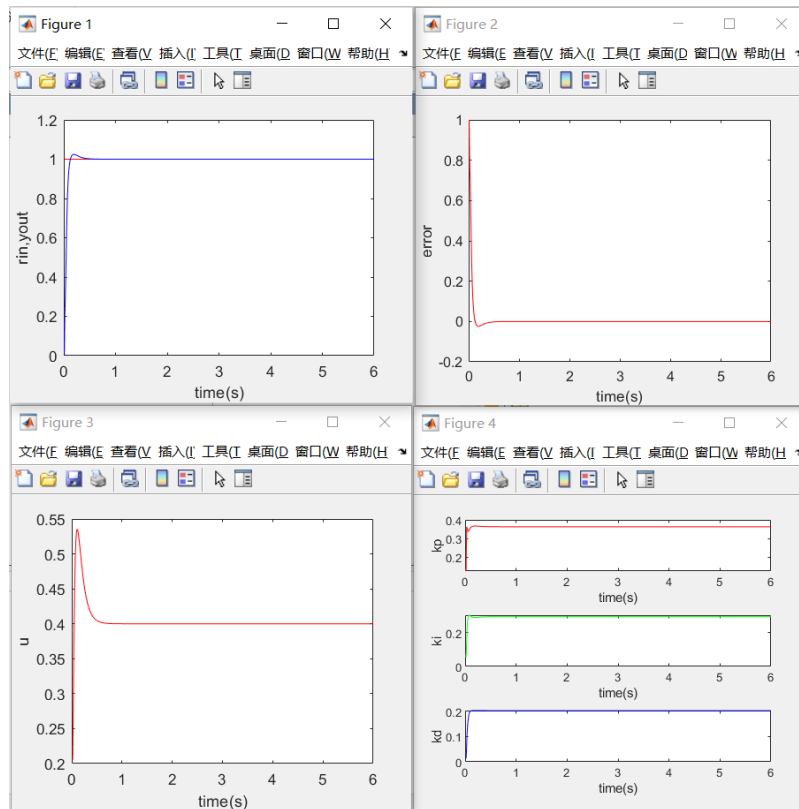
将冲量设置为 0.6：



最大超调量	调节时间(s)	稳态误差
3.8%	0.69	1.9%

观察到冲量很大时，当前输出受前一次影响很大，因此图中有较大类似块状的图形；

将冲量改为 0.2：



最大超调量	调节时间(s)	稳态误差
3.4%	0.45	1.4%

当冲量较小时，前一次输出对当前输出的影响明显变小，这两个图像的对比充分说明了冲量的作用。

3.2.4 分析冲量

冲量 (Momentum) 是一种用于加速训练过程的技术。它通过在梯度更新中引入“速度”或“动量”这一概念，帮助优化算法在训练过程中更快地收敛。冲量的作用类似于物理学中的动量，它使得权重更新方向在一定程度上受到之前更新的影响。

不同情况下，冲量的作用可以总结如下：

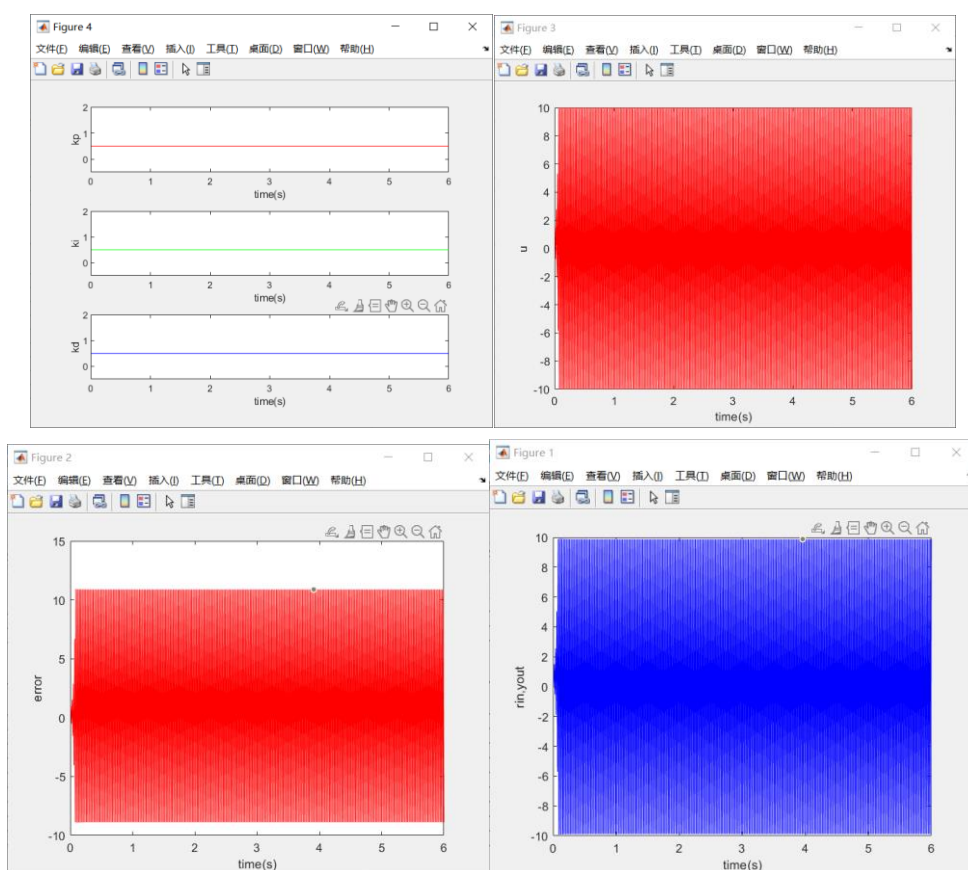
- **冲量的优势：**
 - **加速收敛：** 冲量可以使权重更新在方向上更加稳定，有助于加速收敛。
 - **克服局部极小值：** 冲量有助于克服梯度下降中陷入局部极小值的问题。
- **冲量的劣势：**
 - **选择冲量项：** 冲量项的选择需要仔细调试，不同的问题可能需要不同的冲量项。
 - **可能引入振荡：** 过大的冲量项可能导致模型在参数空间中过度振荡，使得优化不稳定。

3.3 尝试初始权值

3.3.1 全部初始化为 0

初始化方法如下：

```
wi = 0 * ones(H, 4);
wo = 0 * ones(H, 3);
```

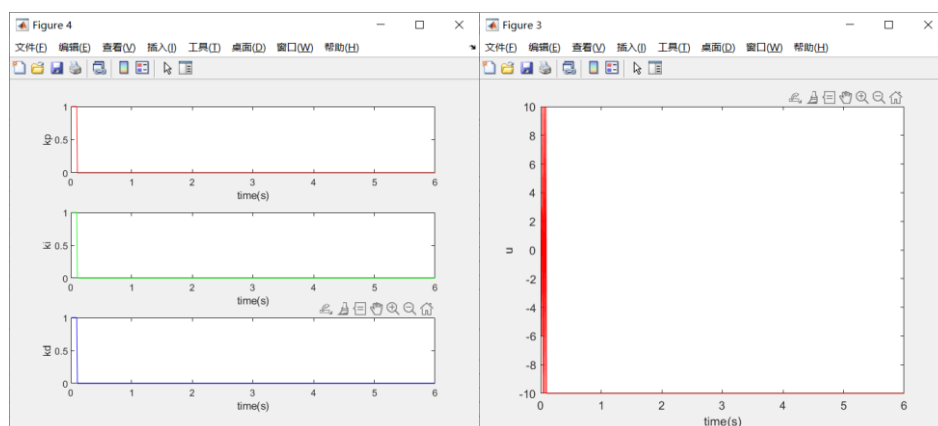


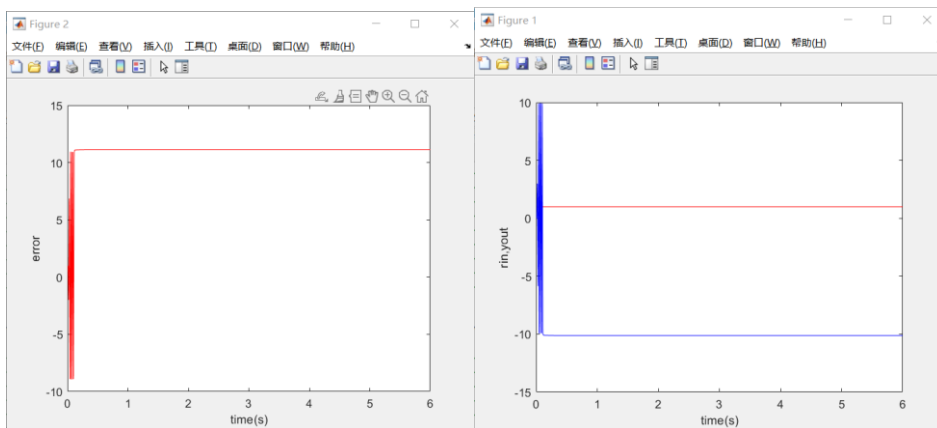
初始化为 0 时优化难度很大，因此振荡严重。

最大超调量	调节时间(s)	稳态误差
无法计算	未到达稳态	未到达稳态

3.3.2 全部初始化为 0.5

初始化方法如 3.2.1

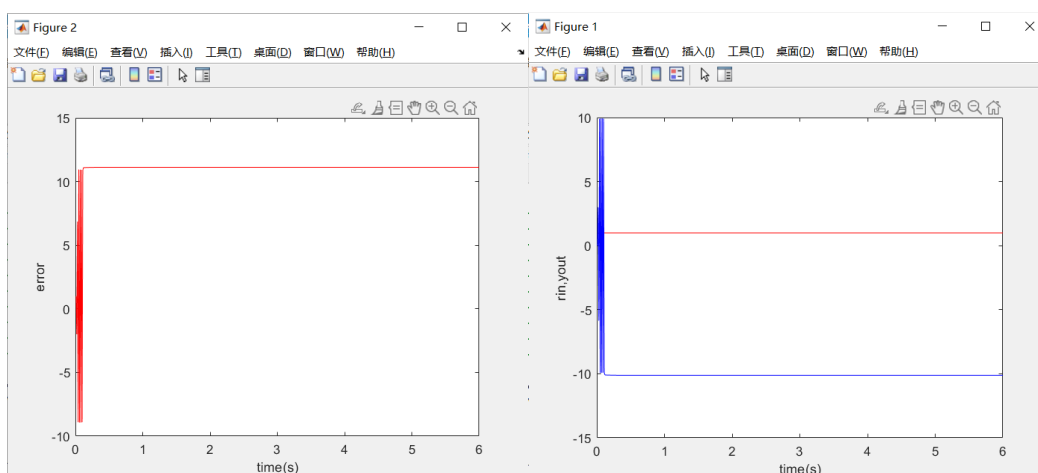
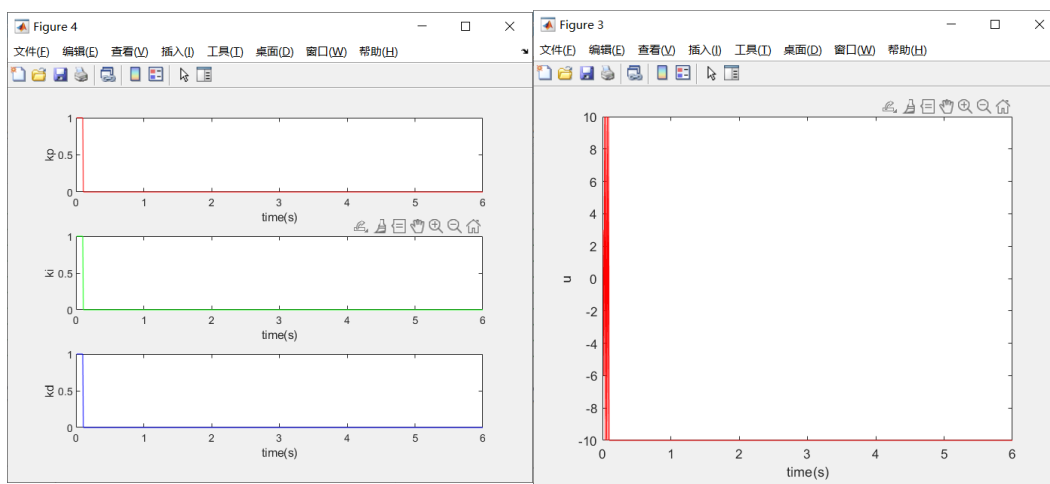




最大超调量	调节时间(s)	稳态误差
200%	0.22	10

初始化为 0.5 时优化效果较差，模型最终没能学会如何正确控制 pid。

3.3.3 全部初始化为-0.5



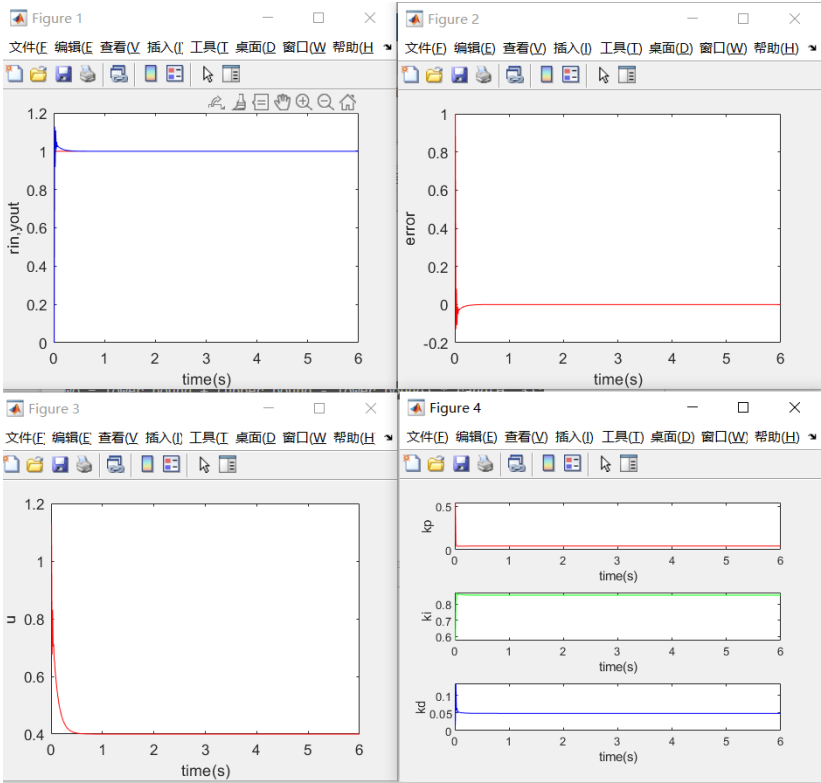
最大超调量	调节时间(s)	稳态误差
200%	0.19	10

效果同初始化为 0.5 时的情况，同初始化为-0.5 的情况，都很差。

3.3.4 初始化为随机数

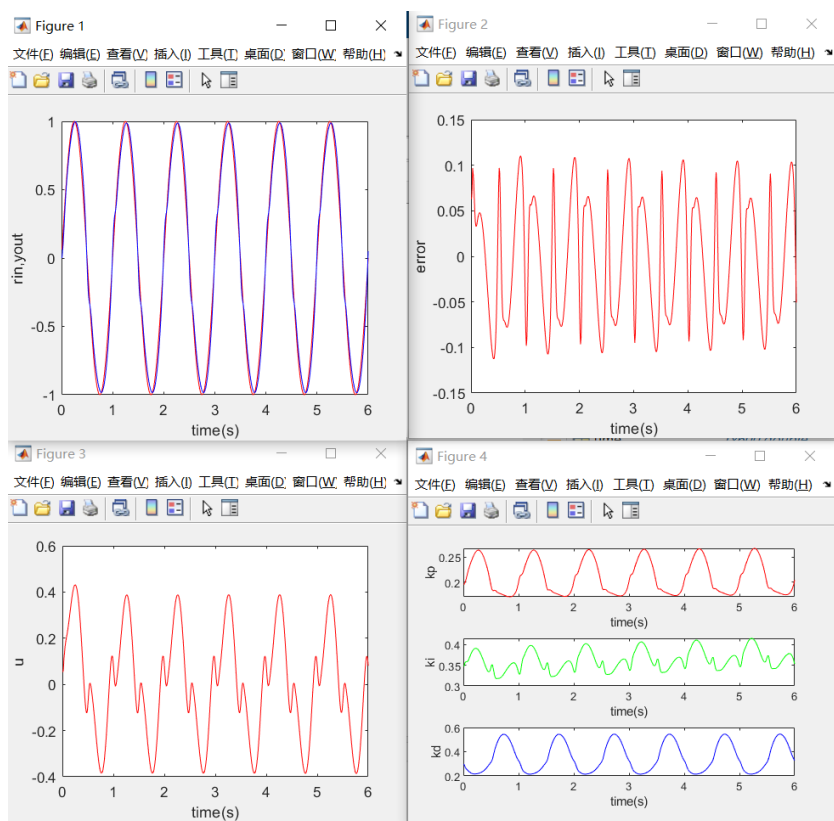
初始化为-0.5 到 0.5 之间的随机数，跟踪曲线设置为 1:

```
upper_bound = 0.5;
lower_bound = -0.5;
wi = lower_bound + (upper_bound - lower_bound) * rand(H, 4);
wo = lower_bound + (upper_bound - lower_bound) * rand(H, 3);
```



最大超调量	调节时间(s)	稳态误差
12%	0.37	3.9%

，使用三角函数作为跟踪曲线，观察到



最大超调量	调节时间(s)	稳态误差
5.7%	0.35	5.5%

3.3.5 初始化分析

初始化权重是神经网络训练中至关重要的一步,不同的初始化方法可能对训练的收敛速度和模型性能产生显著影响。对不同的初始化方法,做如下分析:

预训练权重

- **效果:** 使用预训练的权重,可以提供良好的起始点。
- **问题:** 可能不适用于特定任务或数据集。

随机初始化 (Random Initialization):

- **效果:** 通过从一定范围的均匀或正态分布中随机选择权重,有助于打破对称性,引入多样性。
- **问题:** 选择不当的初始范围可能导致梯度消失或爆炸问题。通常,初始范围的选择要根据激活函数来调整。

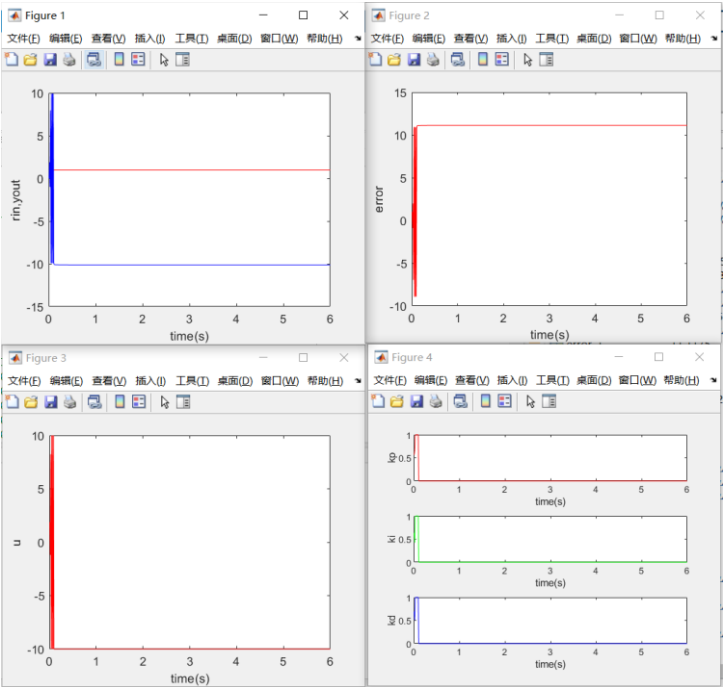
零初始化 (Zero Initialization):

- **效果:** 将所有权重初始化为零可能导致网络对称性问题,每个神经元学到相同的特征,限制了网络的表达能力。
- **问题:** 可能导致梯度消失或爆炸问题。

3.4 尝试 H 个数

跟踪函数为 1, 初始化为 -0.5 到 0.5 之间的随机数:

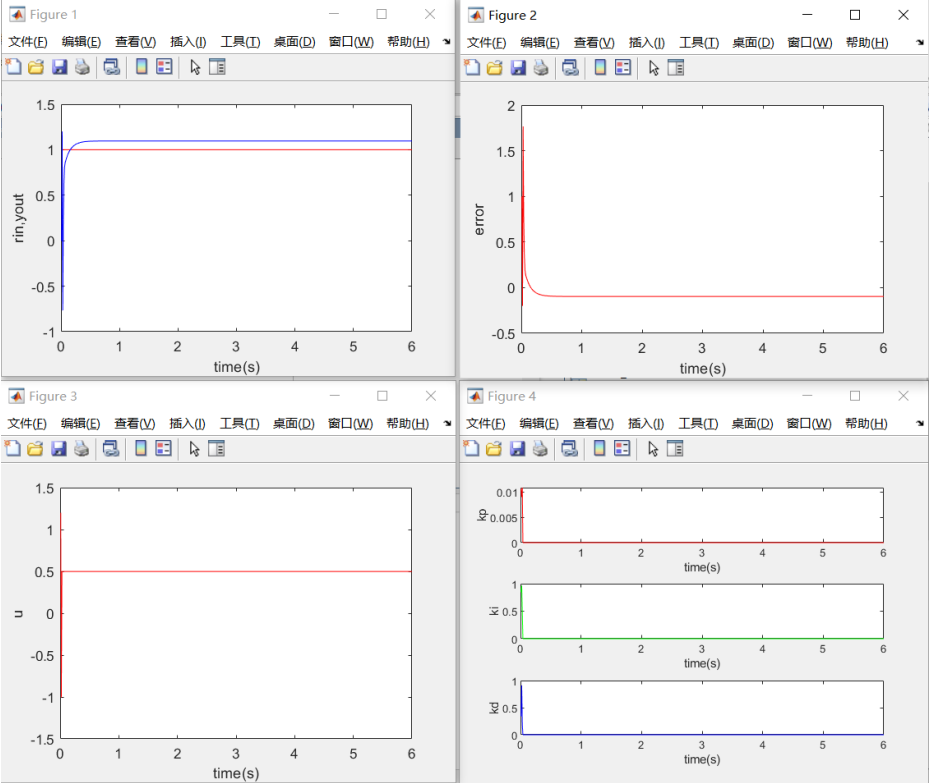
3.4.1 神经元个数为 5:



最大超调量	调节时间(s)	稳态误差
200%	0.18	6.9%

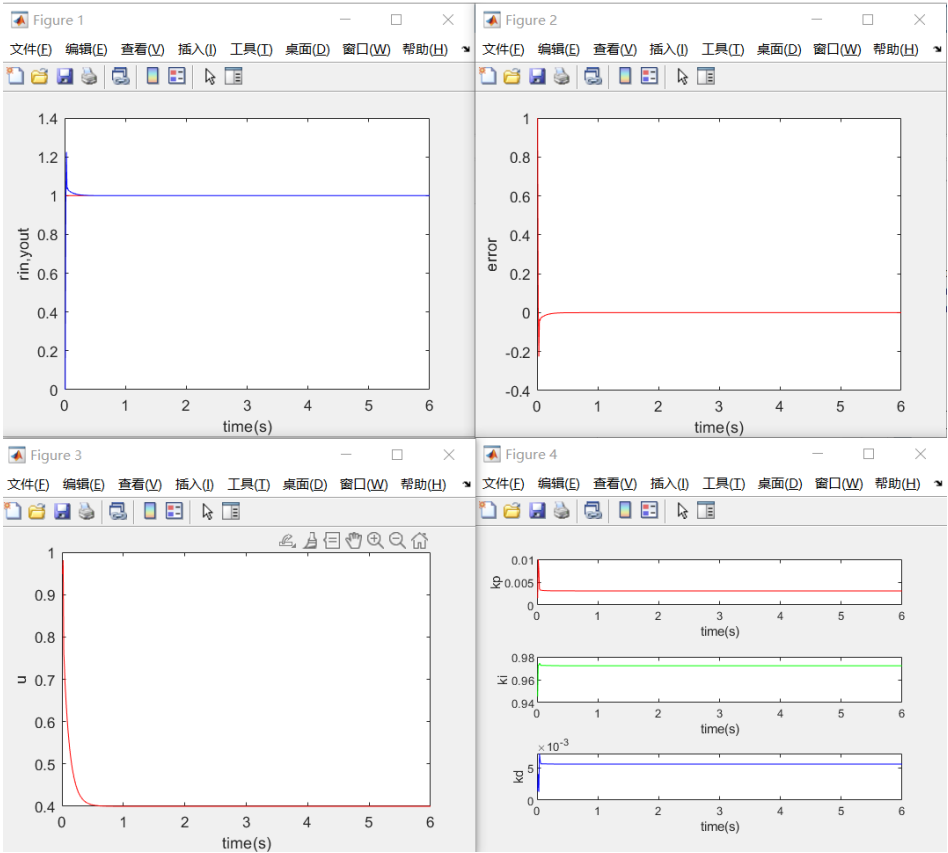
多次尝试均未成功，可见隐藏神经元太少时拟合能力较差。

3.4.2 神经元个数为 20:



最大超调量	调节时间(s)	稳态误差
17.5	0.42	2.1%

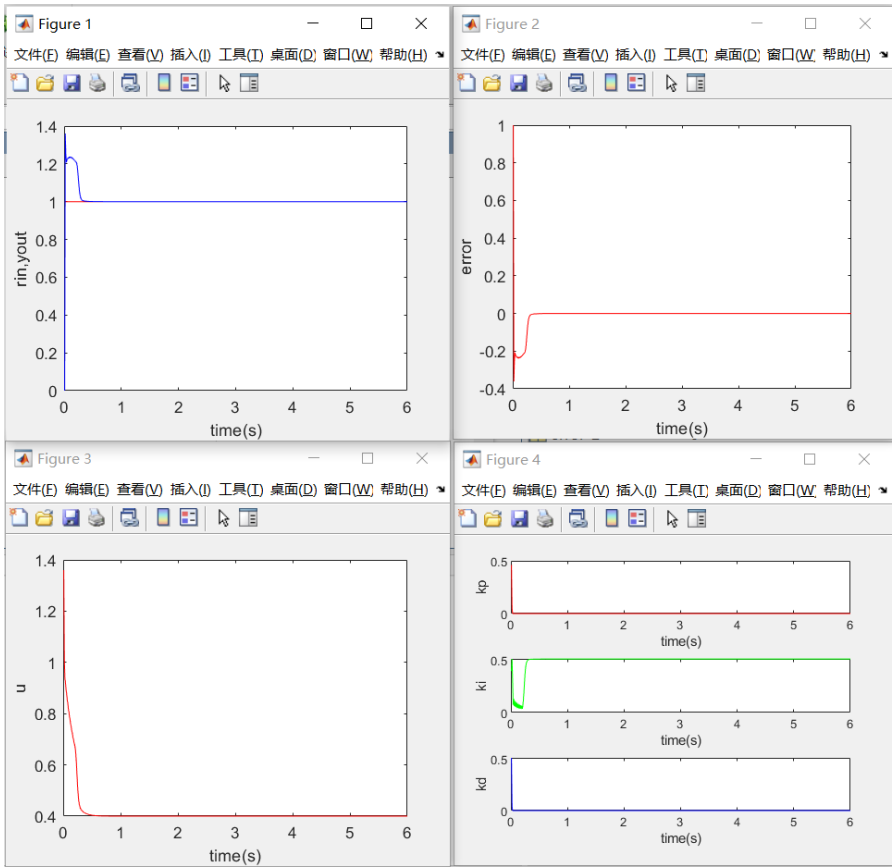
迅速拟合成功。
3.4.3 神经元个数为 50:



最大超调量	调节时间(s)	稳态误差
23%	0.27	3.7%

迅速拟合成功。

3.4.4 神经元个数为 100:



最大超调量	调节时间(s)	稳态误差
37%	0.41	2.2%

迅速拟合成功。

注意到随着神经元个数的增加，pid 参数越来越有效，但当神经元个数过高时，各曲线可能出现抖动，可见单纯地增加隐藏层个数不一定能让模型拟合能力增加，在此基础上，有学者提出了残差神经网络。

3.4.5 神经元个数分析

神经网络的结构，包括神经元个数和层数，是深度学习模型设计中关键的决策因素。

下面是对其展开的一些分析：

- **增加神经元个数的优点：**
 - **更强大的表示能力：** 更多的神经元能够提高网络的表达能力，使其能够学习更复杂的特征和模式。
 - **更好的拟合能力：** 大规模神经网络能够更好地适应训练数据，尤其是在大型数据集上。
- **增加神经元个数的缺点：**
 - **计算和存储成本：** 随着神经元个数的增加，计算和存储需求也会增加，导致训练和推理的时间和资源成本上升。
 - **过拟合的风险：** 如果模型过于复杂，可能会在训练数据上表现很好，但在未见过的数据上泛化能力较差。
- **选择原则：** 根据任务的复杂性和可用的数据量，适度增加神经元个数，以平衡模型复杂度和泛化能力。

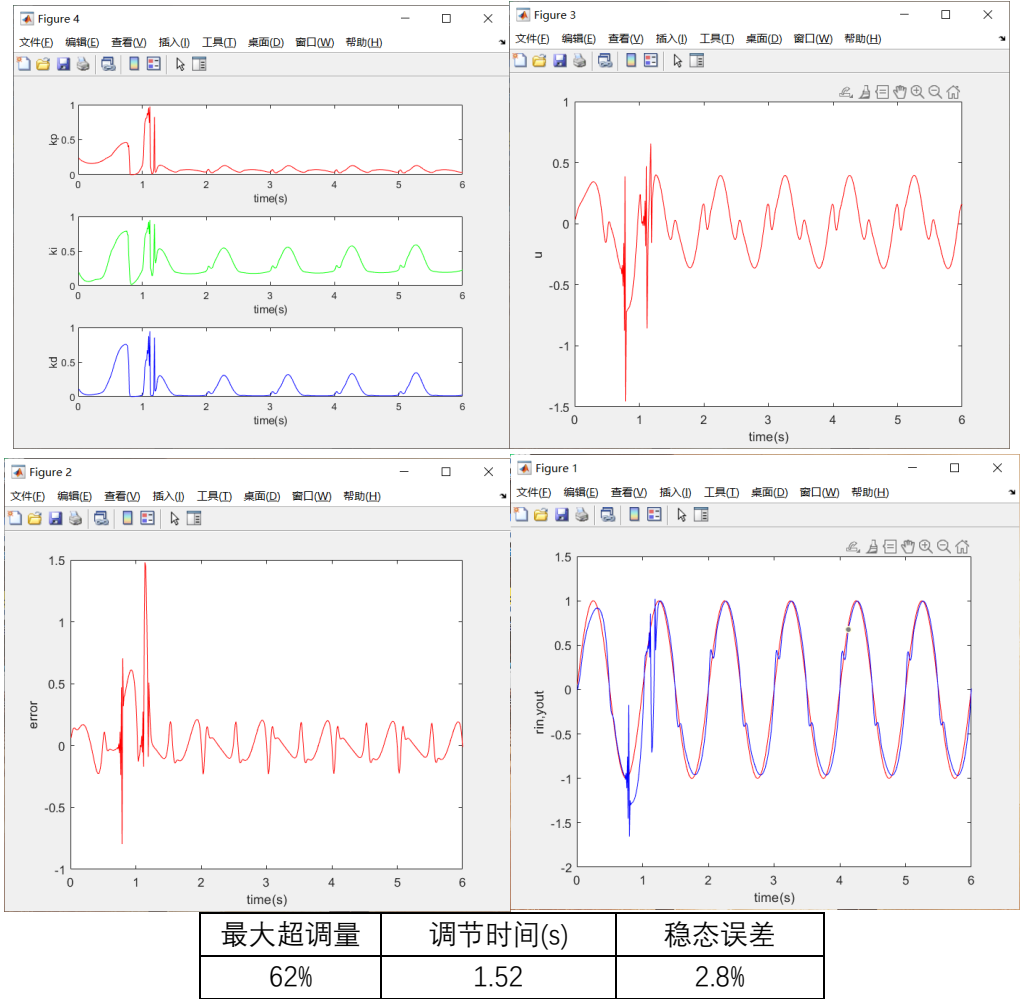
在实验中发现神经元过多不一定能让效果变好,反而使其变差,这一现象正是**残差神经网络 ResNet** 所要解决的痛点问题(当然在这里可能只是因为权重矩阵是初始化产生的,所以才在随机条件下产生了神经元多但效果差的情况)。

3.5 当跟踪曲线是 $r_{in}(k)=\sin(2*3.1415926*time(k))$,是否可以收敛

回答: **可以收敛。**

3.5.1 只改变跟踪曲线

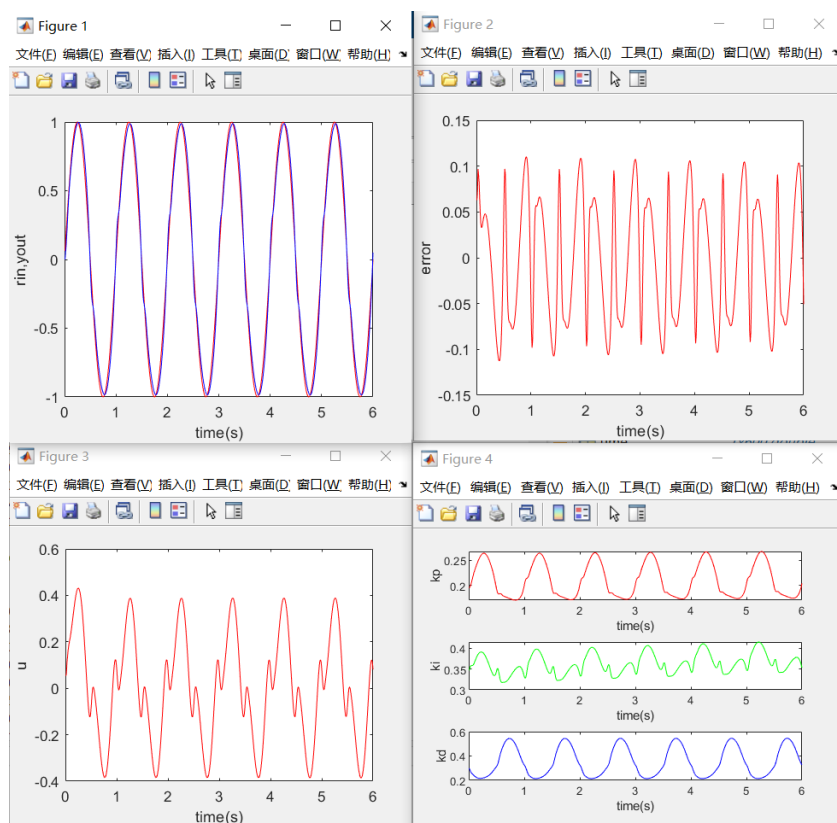
相比初始代码,只改变了跟踪曲线:



观察到神经网络最初还没学到跟踪曲线的规律,所以在第一个波谷到第二个波峰之间误差较大,随着时间推移,神经网络意识到跟踪曲线的规律,所以效果逐渐稳定。

3.5.2 调参

为了让曲线效果更好,我们可以保留前一次权重矩阵作为后一次训练的初始权重矩阵,从而获得更好的效果,观察下图,可以看到曲线非常有规律,拟合非常成功,由此可见好的预训练模型是多么重要:



最大超调量	调节时间(s)	稳态误差
2.6%	0.31	1.6%

3.5.3 拟合效果分析

正弦曲线显然要比为 1 的曲线难拟合得多，所以到达稳态所花的时间也相对更多，但仍然在一层神经网络所能拟合的范围之内。

3.6 两层隐含层

3.6.1 变量间关系及运行流程如下：

此处只给出总体流程，完整代码见附录。

设计第二个隐藏层，设计主要包括初始化、前向和反向过程，此处简述设计思路，具体实现见附录中代码：

初始化

输入层神经元数量 IN 、第一个隐藏层神经元数量 $H1$ 、第二个隐藏层神经元数量 $H2$ 、输出层神经元数量 Out 的定义。

权重矩阵 $wi1$ 、 $wi2$ 分别表示输入层到第一个隐藏层、第一个隐藏层到第二个隐藏层的连接权重。

输出层权重矩阵 $wo1$ 、 $wo2$ 表示第一个隐藏层到输出层、第二个隐藏层到输出层的连接权重。

初始化中间层输出 $Oh1$ 和 $Oh2$ ，以及各层的误差项和历史误差等。

前向与反向

前向与反向传播模仿第一个隐藏层的设计方式，前向传播如下：


```

% 修改中间层的计算（前向传播）
I1 = w11 * xi';
for j = 1:1:H1
    Oh1(j) = (exp(I1(j)) - exp(-I1(j))) / (exp(I1(j)) + exp(-I1(j)));
end

I2 = w12 * Oh1;
for j = 1:1:H2
    Oh2(j) = (exp(I2(j)) - exp(-I2(j))) / (exp(I2(j)) + exp(-I2(j)));
end

% 修改输出层的计算（前向传播）
K = wo * Oh2;
for l = 1:1:Out
    K(l) = exp(K(l)) / (exp(K(l)) + exp(-K(l)));
end

```

反向传播如下：

```

% 更新输出层权重
for l = 1:1:Out
    for i = 1:1:H2
        d_wo = xite * delta3(l) * Oh2(i) + alfa * (wo_1 - wo_2);
    end
end
wo = wo_1 + d_wo + alfa * (wo_1 - wo_2);

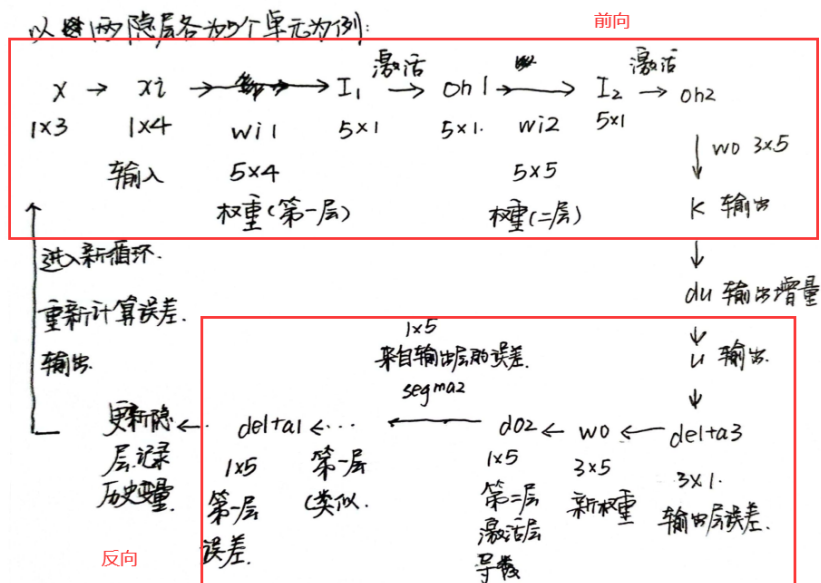
% 修改第二个隐藏层的误差项计算（反向传播）
for i = 1:1:H2
    d02(i) = 4 / (exp(I2(i)) + exp(-I2(i)))^2; % 第二隐藏层的激活层的导数
end

segma2 = delta3 * wo; % 从第三层传回第二层的误差
for i = 1:1:H2
    delta2(i) = d02(i) * segma2(i); % 第二层的更新参考
end

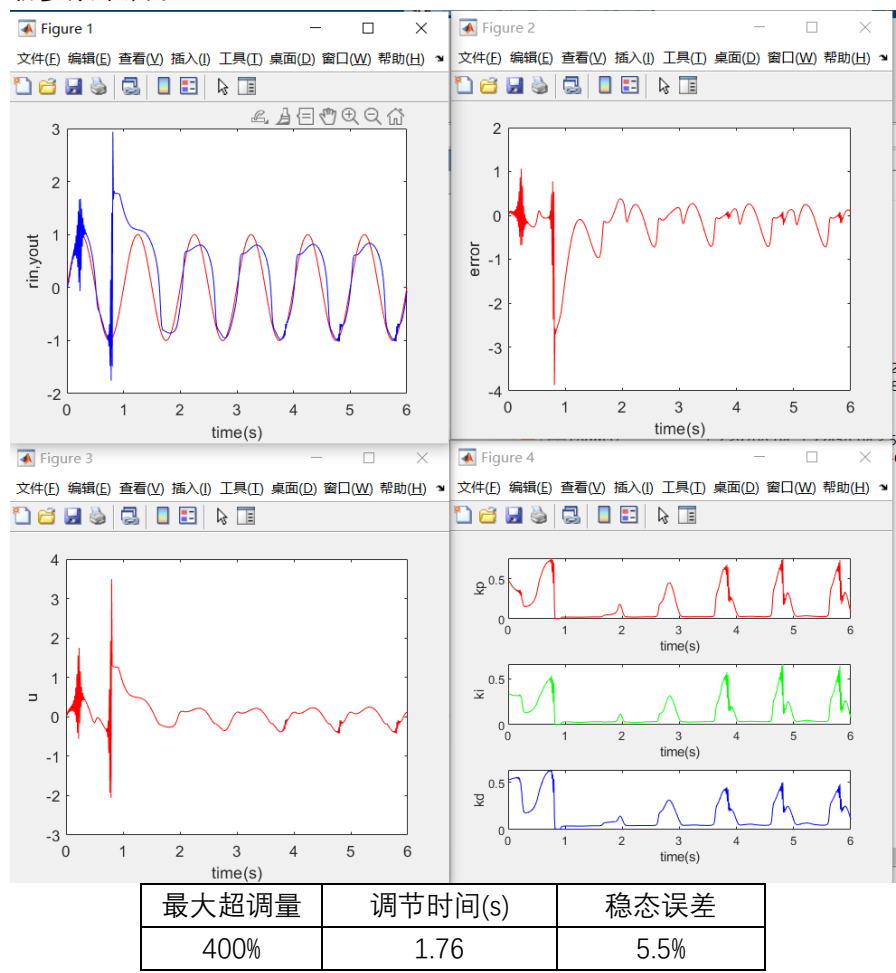
% 修改第一个隐藏层的误差项计算（反向传播）
for i = 1:1:H1
    d01(i) = 4 / (exp(I1(i)) + exp(-I1(i)))^2;
end

```

运行流程如下：

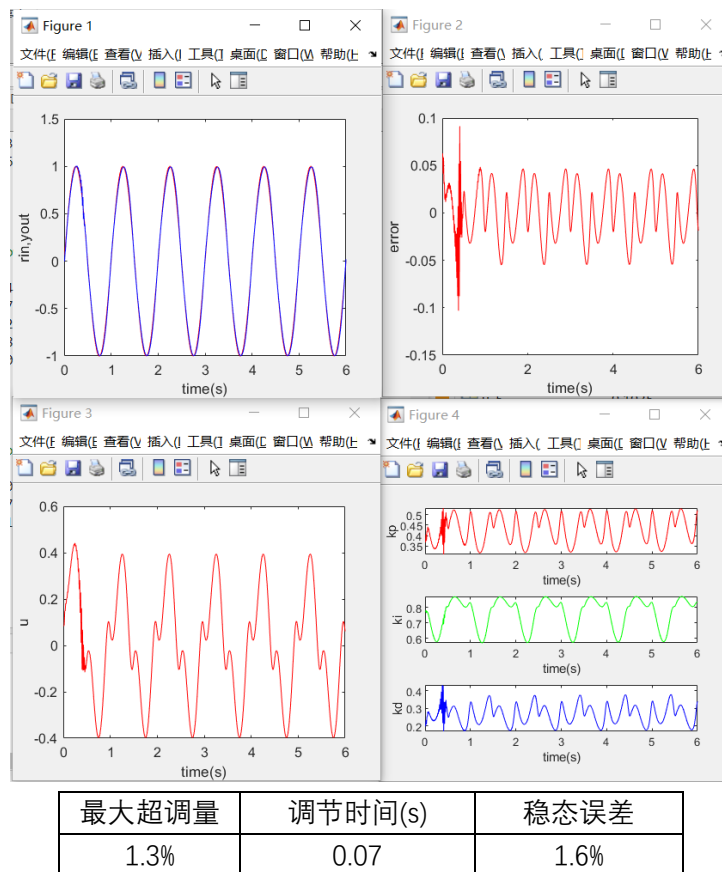


3.6.2 初步效果展示



3.6.3 调参

经过反复调整，将两层神经元个数设置为 5，学习率设置为 0.01，冲量设置为 0.05，保留训练过程中遇见最好的权重矩阵，得到以下结果：



3.6.4 相关分析:

- **增加网络层数的优点:**
 - **更高层次的特征表示:** 深层网络能够逐层学习数据的抽象特征表示, 提高了对数据层次结构的捕捉能力。
 - **更好的非线性变换:** 深层网络能够学习更复杂的非线性映射, 从而更好地拟合复杂的任务。
- **增加网络层数的缺点:**
 - **梯度消失/爆炸问题:** 随着层数的增加, 反向传播中的梯度可能变得非常小或非常大, 导致训练困难。
 - **训练时间和计算成本:** 更深的网络通常需要更长的训练时间和更多的计算资源。
- **选择原则:** 使用合适的初始化、正则化和优化技术来缓解梯度问题, 逐步增加层数并进行实验以监测性能提升。

4 实验总结

本次实验通过神经网络自整定调节 pid, 我对神经网络有了更深刻的理解, 也对 pid 的调节方法有了全新认识, 具体地, 我学会了:

- 使用 matlab 搭建简单神经网络
- 了解了如何初始化权重
- 通过炼丹获得好的初始化权重

5 实验心得

🚩 搭建神经网络

神经网络的重点是数学，只要掌握了推导的原理，用 C++，python，matlab 都可以写出神经网络

🚩 初始化

初始化非常重要，好的初始化权重是反复测试并保留中间结果得到的

🚩 matlab

matlab 对神经网络非常友好，查看矩阵形状非常方便

🚩 超参数

学习率意味着是否“激进”，冲量意味着是否“保守”，神经元个数不是越多越好，个数太多有时不如个数少点，这也是我们提出 ResNet 的原因。

🚩 命名

代码的命名非常重要，源代码的命名方式实在给人造成了很大的阅读困难。

6 附录

🚩 双层神经网络 matlab 代码

```
clear all;
```

```
close all;
```

```
% 设置学习率、惯性项和网络结构参数
```

```
xite = 0.01;%
```

```
alfa = 0.05;%
```

```
IN = 4;
```

```
H1 = 5;
```

```
H2 = 5;
```

```
Out = 3;
```

```
upper_bound = 0.5;
```

```
lower_bound = -0.5;
```

```
% 初始化输入到第一个隐藏层的权重矩阵
```

```
% w11 = rand(H1, IN) * (upper_bound - lower_bound) + lower_bound;
```

```
w11 = [
```

```
0.224521749742638 0.135594038608965 -0.495374905608729 0.370468638276788
```

```
-0.258224317398773 0.208533864549768 -0.410336809224729 0.124555283628723
```

```
0.385945000411898 -0.137524480163592 0.473058602432665 0.624246356204780
```

```
-0.447454200040710 0.465275008730917 -0.405065119763742 -0.392652795823681
```

```
0.0351721811517669 0.409553337425397 -0.0803324923161282 -0.375491804395403
```

```
];
```

```
w11_1 = w11; w11_2 = w11; w11_3 = w11;
```

```
% 初始化第一个隐藏层到第二个隐藏层的权重矩阵
```

```
% w12 = rand(H2, H1) * (upper_bound - lower_bound) + lower_bound;
```

```

wi2 = [
0.631514977480097  0.672016408108704  -0.536551088984277  0.705176693911479
    0.379235619989735
0.333959443821817  0.886744696061373  -0.711445786607146  0.271839754584109
    0.518505808824062
0.642540435060150  0.378662863292834  -0.683195576522955  1.10915289866026
    0.301249182419331
-0.230674713754168 -0.502382511830043  0.113896955089837  -0.673460690088704
    -1.07024971749569
-1.29795170142967  -0.818768274183648  1.69291875204005   -1.53117130552289
    -1.20084542399962
];
wi2_1 = wi2; wi2_2 = wi2; wi2_3 = wi2;

% 初始化第二个隐藏层到输出层的权重矩阵
% wo = rand(Out, H2) * (upper_bound - lower_bound) + lower_bound;
wo = [
1.55284900069045    2.12414592641918    2.21940329986005    1.88136612813388
    1.96797247659794
1.62775433269451    1.61374993496040    1.47187162512370    2.18475673574565
    2.00950920349742
1.99065894211349    2.22757985361920    2.03407910736411    1.86837150950284
    1.65093652507741
];
wo_1 = wo; wo_2 = wo; wo_3 = wo;

% 初始化变量
x = [0, 0, 0];
u_1 = 0; u_2 = 0; u_3 = 0; u_4 = 0; u_5 = 0;
y_1 = 0; y_2 = 0; y_3 = 0;

Oh1 = zeros(H1, 1);
Oh2 = zeros(H2, 1);

error_2 = 0;
error_1 = 0;

ts = 0.01;% time step
for k = 1:1:600
    time(k) = k * ts;
%     rin(k) = 1;% reference input
% 参考输入（这里使用的是步进输入）
% 如果需要使用正弦输入，取消下一行的注释
    rin(k) = sin(2 * pi * time(k));

```

```

a(k) = 1.2 * (1 - 0.8 * exp(-0.1 * k));
yout(k) = a(k) * y_1 / (1 + y_1^2) + u_1;% y_out 实际输出
error(k) = rin(k) - yout(k);% 参考输入（希望的输出）与实际输出的的差
xi = [rin(k), yout(k), error(k), 1];% 神经网络输入

% 计算误差历史
x(1) = error(k) - error_1;
x(2) = error(k);
x(3) = error(k) - 2 * error_1 + error_2;
epid = [x(1); x(2); x(3)];

% 修改中间层的计算（前向传播）
I1 = wi1 * xi';
for j = 1:1:H1
    Oh1(j) = (exp(I1(j)) - exp(-I1(j))) / (exp(I1(j)) + exp(-I1(j)));
end

I2 = wi2 * Oh1;
for j = 1:1:H2
    Oh2(j) = (exp(I2(j)) - exp(-I2(j))) / (exp(I2(j)) + exp(-I2(j)));
end

% 修改输出层的计算（前向传播）
K = wo * Oh2;
for l = 1:1:Out
    K(l) = exp(K(l)) / (exp(K(l)) + exp(-K(l)));
end

% 获取 PID 参数
kp(k) = K(1); ki(k) = K(2); kd(k) = K(3);
Kpid = [kp(k), ki(k), kd(k)];

% 计算控制器输出增量
du(k) = Kpid * epid;
u(k) = u_1 + du(k);

% 限制控制器输出
if u(k) >= 10
    u(k) = 10;
end
if u(k) <= -10
    u(k) = -10;
end

```

```

% 开始反向传播
% 计算误差灵敏度
dyu(k) = sign((yout(k) - y_1) / (u(k) - u_1 + 0.0000001));

% 计算输出层的误差项
for j = 1:1:Out
    dK(j) = 2 / (exp(K(j)) + exp(-K(j)))^2;
end

for l = 1:1:Out
    delta3(l) = error(k) * dyu(k) * epid(l) * dK(l);
end

% 更新输出层权重
for l = 1:1:Out
    for i = 1:1:H2
        d_wo = xite * delta3(l) * Oh2(i) + alfa * (wo_1 - wo_2);
    end
end
wo = wo_1 + d_wo + alfa * (wo_1 - wo_2);

% 修改第二个隐藏层的误差项计算（反向传播）
for i = 1:1:H2
    dO2(i) = 4 / (exp(I2(i)) + exp(-I2(i)))^2;% 第二隐藏层的激活层的导数
end

segma2 = delta3 * wo;% 从第三层传回第二层的误差
for i = 1:1:H2
    delta2(i) = dO2(i) * segma2(i);% 第二层的更新参考
end

% 修改第一个隐藏层的误差项计算（反向传播）
for i = 1:1:H1
    dO1(i) = 4 / (exp(I1(i)) + exp(-I1(i)))^2;
end

segma1 = delta2 * wi2;
for i = 1:1:H1
    delta1(i) = dO1(i) * segma1(i);
end

% 修改权重更新（反向传播）
d_wi2 = xite * delta2' * Oh1' + alfa * (wi2_1 - wi2_2);% 更新量

```

```

wi2 = wi2_1 + d_wi2;

d_wi1 = xite * delta1' * xi + alfa * (wi1_1 - wi1_2);
wi1 = wi1_1 + d_wi1;

% 更新历史参数
u_5 = u_4; u_4 = u_3; u_3 = u_2; u_2 = u_1; u_1 = u(k);
y_2 = y_1; y_1 = yout(k);
wo_3 = wo_2; wo_2 = wo_1; wo_1 = wo;
wi1_3 = wi1_2; wi1_2 = wi1_1; wi1_1 = wi1;
wi2_3 = wi2_2; wi2_2 = wi2_1; wi2_1 = wi2;
error_2 = error_1;
error_1 = error(k);
end

% 绘图
figure(1);
plot(time, rin, 'r', time, yout, 'b');
xlabel('time(s)'); ylabel('rin,yout');

figure(2);
plot(time, error, 'r');
xlabel('time(s)'); ylabel('error');

figure(3);
plot(time, u, 'r');
xlabel('time(s)'); ylabel('u');

figure(4);
subplot(311);
plot(time, kp, 'r');
xlabel('time(s)'); ylabel('kp');

subplot(312);
plot(time, ki, 'g');
xlabel('time(s)'); ylabel('ki');

subplot(313);
plot(time, kd, 'b');
xlabel('time(s)'); ylabel('kd');

```